Bilan

On pensait qu'on aurait juste à reprendre le fichier Python d'embedding du TP précédent en l'ajustant. Mais il y a une grande différence : **embedding.py** faisait une mise à jour statique en base de données entre deux colonnes fixes, tandis que **control_find.py** est une recherche dynamique et interactive qui s'interface en temps réel avec la procédure Oracle pour calculer la similarité.

1. Le problème initial : les limites des embeddings face aux fautes de frappe

L'utilisation des modèles d'embedding (SentenceTransformer) est excellente pour faire de la **recherche sémantique** et comprendre le sens global d'une phrase. Mais on s'est rendu compte que ces modèles ne sont pas conçus pour corriger l'orthographe ou gérer de simples fautes de frappe sur des noms propres

Exemple : Si un utilisateur tape "poser bi", le modèle d'embedding interprète "poser" comme un verbe de la langue française. Il ne fait pas le lien direct avec "Power BI" et renvoie des résultats erronés ou peu pertinents.

2. La stratégie hybride adoptée

Pour contourner ce problème, on a combiné les deux :

- **Garder ce qui avait été fait dans le fichier MFB08** (distances Jaro-Winkler, Q-Gram etc) pour gérer la proximité **textuelle** et les fautes de frappe.

- **Appliquer l'embedding par la suite** (similarité cosinus) pour évaluer la proximité sémantique.

3. Le choix du calcul : Pourquoi la moyenne des scores et non le maximum ?

Dans le script Python, le score final de pertinence est calculé en faisant la **moyenne** des quatre indicateurs de similarité

Pourquoi éviter le maximum ? Si l'on avait pris le score maximum, une seule métrique excellente (par exemple un bon score orthographique isolé) aurait pu écraser et masquer les mauvais résultats des autres indicateurs, provoquant l'affichage de faux positifs.

Pourquoi la moyenne ? Le choix de la moyenne garantit un **consensus équilibré**. Pour qu'une formation apparaisse en tête de liste, elle doit obtenir une validation à la fois sur le plan de la proximité textuelle (correction des fautes) et sur le plan de la cohérence sémantique globale, offrant ainsi des résultats beaucoup plus fiables et précis.

```
  df = pd.read_sql("SELECT * FROM W1", conn)
                          SOUSSOUSDOMAINEDEFORMATION      SCORE
148                                          Power BI  68.457143
150                          Power BI Quelque chose après  45.892823
151  Quelque chose avant Power BI Quelque chose après  40.312500
```